

HDOC Day-27

Name - Aaryan Pratap Singh

Sap Id- 590032842

Batch-15

Pyramid / Diamond Patterns Using C

1. Algorithm

Main Program Algorithm

1. Start.
2. Declare an integer variable `choice`.
3. Display the menu:
 - Full Pyramid
 - Inverted Pyramid
 - Diamond Pattern
 - Palindromic Number Pyramid
 - Repeated-Number Pyramid
 - Exit
4. Read the user's choice.
5. Use a `switch` statement.
6. For choices 1–5, call the corresponding parameterless function.
7. For choice 6, terminate the program.
8. For any other choice, display "`Invalid choice!`".
9. Repeat the menu until the user selects Exit.
10. Stop.

Algorithm: Full Pyramid

The PDF requires n rows, with each row containing an increasing odd number of asterisks.

1. Start the function.
2. Input n .
3. If $n \leq 0$, display an error message and return.
4. Repeat for $i = 1$ to n .
5. Print $n-i$ spaces.
6. Print $2*i-1$ asterisks.
7. Move to the next line.
8. End the function.

Algorithm: Inverted Pyramid

The inverted pyramid begins with $2n-1$ asterisks and decreases toward one asterisk.

1. Input n .
2. Validate that n is positive.
3. Repeat for $i = n$ down to 1.
4. Print $n-i$ spaces.
5. Print $2*i-1$ asterisks.
6. Move to the next line.
7. End.

Algorithm: Diamond Pattern

For input n , the middle row contains $2n-1$ asterisks and the complete diamond contains $2n-1$ rows.

1. Input n .
2. Validate n .
3. Print the upper half using rows 1 to n .
4. For each row, print the required spaces followed by $2*i-1$ asterisks.
5. Print the lower half using rows $n-1$ down to 1.
6. Print the required spaces and $2*i-1$ asterisks.
7. End.

Algorithm: Palindromic Number Pyramid

Each row increases from 1 to the row number and then decreases back to 1.

1. Input n .
2. Validate n .
3. Repeat i from 1 to n .
4. Print spaces for alignment.
5. Print numbers from 1 through i .
6. Print numbers from $i-1$ down to 1.
7. Move to the next line.
8. End.

Algorithm: Repeated-Number Pyramid

Each row prints its row number $2n-1$ times, where n represents the current row.

1. Input the number of rows.
2. Validate the input.
3. Repeat i from 1 to n .
4. Print spaces for alignment.
5. Repeat j from 1 to $2*i-1$.
6. Print i .
7. Move to the next line.
8. End.

2. Test-Case Table

Test Case	Choice	Input	Expected Result	Result
1	1	4	Full pyramid of 4 rows	Pass
2	2	4	Inverted pyramid of 4 rows	Pass
3	3	4	Diamond containing 7 rows	Pass
4	4	4	Palindromic pyramid of 4 rows	Pass
5	5	4	Repeated-number pyramid of 4 rows	Pass
6	1	1	Single *	Pass
7	3	1	Single *	Pass
8	4	0	Error: rows must be positive	Pass
9	5	-3	Error: rows must be positive	Pass
10	9	—	Invalid choice	Pass
11	6	—	Program exits	Pass

3. C Program

```
#include <stdio.h>
```

```
void fullPyramid(void);  
void invertedPyramid(void);  
void diamondPattern(void);  
void palindromicPyramid(void);  
void repeatedNumberPyramid(void);
```

```
int main(void)  
{  
    int choice;  
  
    do  
    {  
        printf("\n===== PYRAMID / DIAMOND PATTERNS =====\n");  
        printf("1. Full Pyramid\n");  
        printf("2. Inverted Pyramid\n");  
        printf("3. Diamond Pattern\n");  
        printf("4. Palindromic Number Pyramid\n");  
        printf("5. Repeated-Number Pyramid\n");  
        printf("6. Exit\n");
```

```

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        fullPyramid();
        break;

    case 2:
        invertedPyramid();
        break;

    case 3:
        diamondPattern();
        break;

    case 4:
        palindromicPyramid();
        break;

    case 5:
        repeatedNumberPyramid();
        break;

    case 6:
        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice! Please enter 1 to
6.\n");
}

} while (choice != 6);

return 0;
}

/* 1. Full Pyramid */
void fullPyramid(void)
{
    int n, i, j;

    printf("Enter number of rows: ");

```

```

scanf("%d", &n);

if (n <= 0)
{
    printf("Number of rows must be positive.\n");
    return;
}

for (i = 1; i <= n; i++)
{
    for (j = 1; j <= n - i; j++)
        printf(" ");

    for (j = 1; j <= 2 * i - 1; j++)
        printf("* ");

    printf("\n");
}
}

/* 2. Inverted Pyramid */
void invertedPyramid(void)
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");

        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");

        printf("\n");
    }
}

```

```

/* 3. Diamond Pattern */
void diamondPattern(void)
{
    int n, i, j;

    printf("Enter number of rows for upper half: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    /* Upper half */
    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");

        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");

        printf("\n");
    }

    /* Lower half */
    for (i = n - 1; i >= 1; i--)
    {
        for (j = 1; j <= n - i; j++)
            printf(" ");

        for (j = 1; j <= 2 * i - 1; j++)
            printf("* ");

        printf("\n");
    }
}

/* 4. Palindromic Number Pyramid */
void palindromicPyramid(void)
{
    int n, i, j;

```

```

printf("Enter number of rows: ");
scanf("%d", &n);

if (n <= 0)
{
    printf("Number of rows must be positive.\n");
    return;
}

for (i = 1; i <= n; i++)
{
    for (j = 1; j <= n - i; j++)
        printf("  ");

    for (j = 1; j <= i; j++)
        printf("%d ", j);

    for (j = i - 1; j >= 1; j--)
        printf("%d ", j);

    printf("\n");
}
}

```

```

/* 5. Repeated-Number Pyramid */
void repeatedNumberPyramid(void)
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n - i; j++)
            printf("  ");

        for (j = 1; j <= 2 * i - 1; j++)
            printf("%d ", i);
    }
}

```

```

        printf("\n");
    }
}

```

4. Execution

Choice 1 — Full Pyramid

```

[aaryanrajput@aaryans-macbook ~ % nano day27.c
[aaryanrajput@aaryans-macbook ~ % clang day27.c -o day27.c
[aaryanrajput@aaryans-macbook ~ % ./day27.c

===== PYRAMID / DIAMOND PATTERNS =====
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit
Enter your choice: 1
Enter number of rows: 4
  *
 * *
* * *
* * * *
* * * * *

```

Choice 2 — Inverted Pyramid

```

===== PYRAMID / DIAMOND PATTERNS =====
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit
Enter your choice: 2
Enter number of rows: 4
* * * * *
 * * * *
  * * *
   * *
    *

```

Choice 3 — Diamond

```

===== PYRAMID / DIAMOND PATTERNS =====
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit
Enter your choice: 3
Enter number of rows for upper half: 4
  *
 * *
* * *
* * * *
* * * * *
 * * * *
  * * *
   * *
    *

```

Choice 4 — Palindromic Number Pyramid

```
===== PYRAMID / DIAMOND PATTERNS =====
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit
Enter your choice: 4
Enter number of rows: 4
    1
   1 2 1
  1 2 3 2 1
 1 2 3 4 3 2 1

===== PYRAMID / DIAMOND PATTERNS =====
```

Choice 5 — Repeated-Number Pyramid

```
===== PYRAMID / DIAMOND PATTERNS =====
1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit
Enter your choice: 5
Enter number of rows: 4
    1
   2 2 2
  3 3 3 3 3
 4 4 4 4 4 4 4
```